

PROGRAMAÇÃO ORIENTADA A OBJETOS COM BANCO DE DADOS



Atributos e métodos de classe

Victor de Andrade Machado

OBJETIVOS DE APRENDIZAGEM

- > Identificar atributos de instância e de classe ou estáticos.
- > Definir métodos de instância e de classes ou estáticos.
- > Construir classes com métodos e atributos de classes ou estáticos.

Introdução

Na programação orientada a objetos, os atributos e métodos desempenham um papel fundamental na definição do comportamento e das características de uma classe. Em Java, uma das linguagens de programação mais populares e amplamente utilizadas, os atributos representam as propriedades ou características de um objeto, enquanto os métodos são responsáveis por executar ações relacionadas a esse objeto.

Neste capítulo, você estudará em detalhes os atributos e métodos em classes de programação na linguagem Java. Abordaremos os diferentes tipos de atributos, como atributos de instância e atributos de classe (ou estáticos). Além disso, discutiremos os diferentes tipos de métodos, como métodos de instância e métodos de classe (ou estáticos). Ao longo deste capítulo, também mostraremos exemplos práticos de construção de classes com esses atributos e métodos.

Conceitos e tipos de atributos

Os atributos em Java representam as características ou propriedades de um objeto. Numa classe, existem dois tipos principais de atributos: atributos de instância e atributos de classe (DEITEL; DEITEL, 2018).

Os atributos de instância são variáveis declaradas dentro de uma classe, mas fora de qualquer método. Cada instância (ou objeto) criada a partir dessa classe terá sua própria cópia desses atributos, ou seja, cada objeto possui seu conjunto separado de valores para esses atributos (FINEGAN; LIGUORI, 2018). Esses atributos são definidos usando o modificador de acesso (`public`, `private`, `protected` ou sem modificador) seguido do tipo de dado e do nome do atributo.

Para definir um atributo de instância, declaramos a variável dentro da classe, mas fora de qualquer método, construtor ou bloco estático (HORSTMANN, 2008), conforme mostrado na Figura 1.

```
public class Carro {  
    String marca;  
    String modelo;  
}
```

Figura 1. Definição de atributo de instância em Java.

Nesse exemplo, `marca` e `modelo` são atributos de **instância** da classe `Carro`. Cada objeto criado a partir dessa classe terá seus próprios valores para esses atributos.

Por outro lado, os atributos de classe (ou estáticos) são compartilhados por todas as instâncias da classe. Eles pertencem à classe em si e não a objetos individuais. Para definir um atributo de classe, usamos a palavra-chave `static` (DEITEL; DEITEL, 2018).

Os atributos de **classe** são úteis quando você deseja armazenar informações que são comuns a todas as instâncias da classe, como uma variável de contagem ou uma constante. Um exemplo é mostrado na Figura 2.

```
public class Carro {  
    String marca;  
    String modelo;  
    static String fabricante; //atributo de classe  
}
```

Figura 2. Exemplo de atributo de classe em Java.

No código fornecido, temos a definição da classe `Carro`. Essa classe possui três atributos: `marca`, `modelo` e `fabricante`. No entanto, o atributo `fabricante` se destaca por ser declarado como estático, utilizando a palavra-chave `static`. Esse é um exemplo de atributo de classe.

O atributo de classe `fabricante` pode ser acessado tanto pelas instâncias da classe quanto diretamente pela própria classe, sem a necessidade de criar objetos. Ele é compartilhado por todas as instâncias e pertence à classe como um todo. Para acessar o atributo de classe `fabricante`, podemos utilizar o nome da classe seguido do nome do atributo. Por exemplo:

```
Carro.fabricante = "Toyota";
```

Nesse exemplo, definimos o fabricante como `Toyota` para todos os carros da classe `Carro`. Essa alteração será refletida em todas as instâncias da classe.

É importante destacar que, ao utilizar um atributo de classe, devemos ter cuidado ao modificá-lo, uma vez que a alteração afetará todas as instâncias da classe. Por isso, é recomendável utilizá-lo com responsabilidade e de maneira adequada ao contexto do aplicativo em questão (FINEGAN; LIGUORI, 2018).

Compreender o impacto das modificações em atributos de classe é essencial para o desenvolvimento responsável de aplicativos em Java. A consideração cuidadosa dessas alterações permite evitar efeitos indesejados em todas as instâncias da classe. Ao adotar uma abordagem criteriosa e contextualizada no uso dos atributos de classe, é possível garantir a consistência e o bom funcionamento do código. Na próxima seção, exploraremos os métodos de instância e de classe em Java, aprofundando ainda mais nosso conhecimento sobre como utilizar esses recursos de forma eficiente e eficaz

Métodos de instância e de classe em Java

Os métodos em Java são responsáveis por executar ações relacionadas a um objeto ou à classe em si. Assim como os atributos, existem dois tipos principais de métodos: métodos de instância e métodos de classe (ou estáticos).

Os **métodos de instância** estão associados a objetos específicos de uma classe. Eles são definidos dentro da classe, mas só podem ser invocados por meio de uma instância daquela classe. Cada objeto criado a partir de uma classe tem sua própria cópia dos métodos de instância. Isso significa que cada objeto pode chamar e executar esses métodos independentemente uns dos outros (SIERRA; BATES, 2019).



Fique atento

Dentro de um método de instância, você pode acessar diretamente os atributos (variáveis) da classe, bem como outros métodos de instância.

Para chamar um método de instância, você precisa criar uma instância (objeto) da classe e usar a notação de ponto para invocar o método na instância desejada. Por exemplo:

```
objeto.metodoDeInstancia().
```

Os métodos de instância são usados principalmente para realizar operações ou manipulações específicas em um objeto individual. Uma classe `Carro`, por exemplo, pode ter um método de instância chamado `acelerar()` que aumenta a velocidade do carro específico representado por um objeto da classe. Veja na Figura 3 um exemplo de implementação em Java de método de instância.

```
public class Carro {  
  
    private String marca;  
    private String modelo;  
    private int ano;  
  
    // Construtor da classe Carro  
    public Carro(String marca, String modelo, int ano) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
    }  
  
    // Método de instância que imprime as informações do carro  
    public void imprimirInformacoes() {  
        System.out.println("Marca: " + marca);  
        System.out.println("Modelo: " + modelo);  
        System.out.println("Ano: " + ano);  
    }  
  
    public static void main(String[] args) {  
        // Criando uma instância da classe Carro  
        Carro meuCarro = new Carro("Toyota", "Corolla", 2021);  
  
        // Chamando o método de instância para imprimir as informações  
do carro  
        meuCarro.imprimirInformacoes();  
    }  
}
```

Figura 3. Método de instância em Java.

Nesse exemplo, temos a classe `Carro` com um método de instância chamado `imprimirInformacoes()`. Esse método é responsável por exibir as informações do carro, como marca, modelo e ano. Ao criar uma instância da classe `Carro` e chamar o método `imprimirInformacoes()` nessa instância, as informações específicas do carro são impressas. A saída do programa pode ser vista na Figura 4. Observe que a instância criada com a chamada do programa exibe seus valores, garantindo que cada objeto tenha sua própria identidade.

```
Marca: Toyota  
Modelo: Corolla  
Ano: 2021
```

Figura 4. Saída do programa.

Por sua vez, os **métodos de classe** são associados à classe em si, não a instâncias ou objetos específicos dessa classe. Eles são definidos usando a palavra-chave `static` e podem ser invocados diretamente na classe, sem a necessidade de criar uma instância.

Ao contrário dos métodos de instância, os métodos de classe não têm acesso direto aos atributos de instância da classe. Isso ocorre porque não são vinculados a objetos específicos e não têm acesso aos dados de um objeto específico.



Saiba mais

Os métodos de classe são úteis quando você deseja executar uma operação que não depende de um estado específico do objeto. Por exemplo, um método de classe numa classe `Calculadora` poderia ser `somar()`, que recebe dois números como argumentos e retorna sua soma. Não é necessário ter um objeto `Calculadora` específico para realizar essa operação.

Para chamar um método de classe, você utiliza a notação de ponto diretamente na classe, sem criar uma instância. Por exemplo:

```
NomeDaClasse.metodoDeClasse().
```

Os métodos de classe são frequentemente usados para criar funções utilitárias, como conversores de unidades, validadores de entrada, métodos matemáticos estáticos, entre outros. Veja um exemplo mostrado na Figura 5.

```
public class Calculadora {

    public static int somar(int num1, int num2) {
        return num1 + num2;
    }

    public static void main(String[] args) {
        int resultado = Calculadora.somar(5, 3);
        System.out.println("Resultado: " + resultado);
    }
}
```

Figura 5. Método de classe em Java.

Ao ser executado, a seguinte saída será exibida:

```
Resultado: 8
```

Nesse exemplo, temos a classe `Calculadora` com um método de classe chamado `somar()`. Esse método recebe dois números como parâmetros e retorna a soma desses números. Não é necessário criar uma instância da classe `Calculadora` para chamar o método `somar()`. Podemos chamar diretamente o método usando a notação `NomeDaClasse.metodoDeClasse()`. No exemplo, chamamos o método `somar(5, 3)` e atribuímos o resultado a uma variável chamada `resultado`, que é então impressa.

Esses exemplos ilustram a diferença entre métodos de instância e métodos de classe em Java, mostrando como são definidos, chamados e utilizados em diferentes contextos. Assim, podemos ver a seguir um exemplo mais completo utilizando atributos e métodos de instância e de classe.

Construção de métodos e atributos em Java

Para demonstrar a utilização dos atributos e métodos, vamos criar um exemplo composto por duas classes básicas: `Carro` e `Frota`. A classe `Carro` representará um carro individual, enquanto a classe `Frota` será responsável por manter o controle de uma frota de carros. O código da classe `Carro` é mostrado na Figura 6.

```
public class Carro {  
  
    private static int totalCarros; // atributo de classe  
    private String marca; // atributo de instância  
    private String modelo; // atributo de instância  
  
    public Carro(String marca, String modelo) {  
        this.marca = marca;  
        this.modelo = modelo;  
        totalCarros++;  
    }  
  
    public String getMarca() {  
        return marca;  
    }  
  
    public String getModelo() {  
        return modelo;  
    }  
  
    public static int getTotalCarros() {  
        return totalCarros;  
    }  
}
```

Figura 6. Código da classe Carro.

Analisamos o código da Figura 6 conforme as descrições a seguir:

- Atributo de classe `totalCarros`: é um atributo estático que mantém o controle do número total de carros criados.
- Atributos de instância `marca` e `modelo`: representam a marca e o modelo de um carro.
- Construtor: recebe a marca e o modelo do carro e atribui-os aos respectivos atributos de instância. Além disso, incrementa o contador `totalCarros` em 1.
- Métodos de instância `getMarca()` e `getModelo()`: retornam a marca e o modelo do carro, respectivamente.
- Método de classe `getTotalCarros()`: retorna o número total de carros criados.

O próximo código, mostrado na Figura 7, é referente à classe `Frota`.

```
public class Frota {  
    private Carro[] carros; // atributo de instância  
  
    public Frota(int tamanho) {  
        carros = new Carro[tamanho];  
    }  
  
    public void adicionarCarro(Carro carro, int indice) {  
        carros[indice] = carro;  
    }  
  
    public Carro getCarro(int indice) {  
        return carros[indice];  
    }  
}
```

Figura 7. Código da Classe `Frota`.

Nesse código, temos as seguintes características:

- Atributo de instância `carros`: é um *array* de objetos da classe `Carro` e armazena os carros da frota.
- Construtor: recebe o tamanho da frota como parâmetro e inicializa o *array* `carros` com o tamanho especificado.
- Método de instância `adicionarCarro()`: recebe um objeto da classe `Carro` e um índice como parâmetros, e adiciona o carro ao *array* `carros` na posição indicada pelo índice.
- Método de instância `getCarro()`: recebe um índice como parâmetro e retorna o carro do *array* `carros` na posição indicada pelo índice.

Para a utilização correta das classes `Carro` e `Frota`, construímos uma classe principal chamada de `Main`, cujo código é mostrado na Figura 8.

```

public static void main(String[] args) {
    Frota frota = new Frota(3);

    Carro carro1 = new Carro("Ford", "Fiesta");
    Carro carro2 = new Carro("Chevrolet", "Cruze");
    Carro carro3 = new Carro("Volkswagen", "Golf");

    frota.adicionarCarro(carro1, 0);
    frota.adicionarCarro(carro2, 1);
    frota.adicionarCarro(carro3, 2);

    System.out.println("Total de carros: " +
Carro.getTotalCarros());

    Carro[] carrosFrota = frota.getCarros();
    for (int i = 0; i < carrosFrota.length; i++) {
        Carro carro = carrosFrota[i];
        System.out.println("Carro " + (i + 1) + ": " +
carro.getMarca() + " " + carro.getModelo());
    }
}
}

```

Figura 8. Classe principal em Java.

A classe, aqui chamada de `Main`, possui o método principal, ou `main`, para realizar as chamadas de execução do programa. Sua descrição é feita a seguir:

- O método `main()` é o ponto de entrada do programa.
- Uma instância da classe `Frota` chamada `frota` é criada, com tamanho 3.
- Três instâncias da classe `Carro` são criadas, cada qual com uma marca e modelo diferentes.
- Os carros são adicionados à frota usando o método `adicionarCarro()` da classe `Frota`, passando o objeto `Carro` e o índice de posição desejada.
- O número total de carros é impresso utilizando o método de classe `getTotalCarros()` da classe `Carro`.
- O método `getCarros()` da classe `Frota` é utilizado para obter o array de carros da frota.

- Num *loop for*, cada carro do *array* `carrosFrota` é recuperado e armazenado numa variável chamada `carro`.
- Em cada iteração do *loop*, a marca e o modelo do carro são impressos utilizando os métodos `getMarca()` e `getModelo()` da classe `Carro`.

A saída do programa é vista na Figura 9, mostrada a seguir.

```
Total de carros: 3
Carro 1: Ford Fiesta
Carro 2: Chevrolet Cruze
Carro 3: Volkswagen Golf
```

Figura 9. Saída após execução da classe principal em Java.

O programa itera sobre os carros da frota usando o método `getCarros` da classe `Frota` para obter a matriz de carros e, em seguida, exibe a marca e o modelo de cada carro no console. A saída mostra que foram criados três objetos de carro e que eles foram adicionados à frota corretamente, mantendo a ordem em que foram adicionados.

Neste capítulo, abordamos os conceitos de atributos e métodos de instância e de classe em Java. Os atributos de instância representam características individuais de cada objeto, enquanto os atributos de classe são compartilhados por todas as instâncias da classe (SCHILDT, 2015). Os métodos de instância operam em objetos específicos e têm acesso aos atributos de instância, enquanto os métodos de classe operam na classe como um todo, sem depender de instâncias específicas.

Esses conceitos são fundamentais para o desenvolvimento de programas orientados a objetos em Java, permitindo a criação de objetos com comportamentos e características distintas, além de facilitar o compartilhamento de informações entre as instâncias da mesma classe. Compreender esses conceitos é essencial para criar programas mais estruturados, flexíveis e reutilizáveis.

Referências

DEITEL, P.; DEITEL, H. *Java: como programar*. 11. ed. New York: Pearson, 2018.

FINEGAN, E.; LIGUORI, R. *OCA Java SE 8: guia de estudos para o exame 1Z0-808*. Porto Alegre: Bookman, 2018.

HORSTMANN, C. *Conceitos de computação com Java: compatível com Java 5 & 6*. 5. ed. Porto Alegre: Bookman, 2008.

SCHILD, H. *Java para iniciantes: crie, compile e execute programas Java rapidamente*. 6. ed. Porto Alegre: Bookman, 2015.

SIERRA, K.; BATES, B. *Use a cabeça! Java*. Rio de Janeiro: Alta Books, 2019.

Leituras recomendadas

GOODRICH, M. T.; TAMASSIA, R. *Estruturas de dados e algoritmos em Java*. 5. ed. Porto Alegre: Bookman, 2013.

SCHACH, S. R. *Engenharia de software: os paradigmas clássico e orientado a objetos*. 7. ed. Porto Alegre: AMGH, 2010.

Conteúdo:

sagah⁺